

IF2224 FORMAL LANGUAGE AND AUTOMATA THEORY

Arion Interpreter — Milestone 1: Lexical Analysis

March 29, 2026

Made Branenda Jordhy — 13524026

Muhammad Nur Majiid — 13524028

Jason Edward Salim — 13524034

Athilla Zaidan Zidna Fann — 13524068

**School of Electrical Engineering and Informatics
Institut Teknologi Bandung
Semester II 2025/2026**

Contents

1	Theoretical Background	3
1.1	Lexical Analysis	3
1.2	Token	3
1.3	Deterministic Finite Automaton (DFA)	3
2	Design and Implementation	4
2.1	Overview	4
2.2	DFA Design	4
2.3	Tokenization Flow	7
2.4	Token Classification	7
3	Testing	9
4	Conclusion	9
	Appendix	10

1 Theoretical Background

1.1 Lexical Analysis

Lexical analysis adalah fase pertama dalam pipeline kompilasi maupun interpretasi. Fase ini bertanggung jawab mengubah aliran karakter mentah dari source code menjadi aliran *token* berstruktur yang siap dikonsumsi oleh parser. Komponen yang melaksanakan proses ini disebut *lexer* atau *scanner*.

Lexer beroperasi dengan membaca input secara sekuensial, satu karakter per iterasi, sambil memelihara state internal yang merepresentasikan pola yang sedang dikenali. Karakter yang tidak membawa makna semantis, yaitu whitespace dan komentar, dibuang pada fase ini sehingga tidak membebani tahap analisis selanjutnya. Keluaran lexer adalah barisan token yang menjadi representasi leksikal dari program, dan barisan inilah yang diserahkan ke *syntax analyzer* untuk diproses lebih lanjut.

1.2 Token

Sebuah *token* adalah pasangan (t, v) di mana t menyatakan *token type* dan v menyatakan *lexeme*, yaitu substring dari source code yang membentuk token tersebut. Sebagai contoh, ekspresi `x := 5` menghasilkan tiga token: `IDENT("x")`, `BECOMES`, dan `INTCON("5")`.

Token dalam bahasa pemrograman umumnya terklasifikasi ke dalam kategori berikut.

- **Literals.** Konstanta bernilai tetap: integer, bilangan riil, karakter, dan string.
- **Keywords.** Reserved words yang memiliki semantik khusus dalam bahasa, seperti `if`, `while`, `begin`, dan `function`.
- **Identifiers.** Nama yang didefinisikan pengguna untuk variabel, fungsi, prosedur, dan tipe.
- **Operators.** Simbol operasi aritmatika, logika, dan relasional.
- **Delimiters.** Karakter pembatas struktural seperti `(`, `)`, `;`, dan `..`

Perlu diperhatikan bahwa keyword dan identifier secara leksikal memiliki pola reguler yang identik. Disambiguasi dilakukan setelah lexeme terbaca secara lengkap: lexeme yang terdaftar sebagai reserved word dikategorikan sebagai keyword, selainnya sebagai identifier.

1.3 Deterministic Finite Automaton (DFA)

Pola leksikal dari setiap kategori token dapat dideskripsikan oleh *regular expression*, dan setiap regular expression ekuivalen dengan sebuah *finite automaton*. Model komputasi yang digunakan dalam implementasi lexer adalah *Deterministic Finite Automaton* (DFA).

Definisi. Sebuah DFA adalah 5-tupel $M = (Q, \Sigma, \delta, q_0, F)$, di mana: Q adalah himpunan state berhingga, Σ adalah alfabet input, $\delta : Q \times \Sigma \rightarrow Q$ adalah fungsi transisi total, $q_0 \in Q$ adalah state awal, dan $F \subseteq Q$ adalah himpunan accepting states.

Sifat deterministik dari DFA dinyatakan oleh totalitas δ : untuk setiap $(q, a) \in Q \times \Sigma$, terdapat tepat satu state $q' = \delta(q, a)$. Hal ini menjamin bahwa eksekusi DFA pada string input manapun bersifat unik dan dapat diprediksi, tanpa backtracking maupun pencabangan komputasi.

Dalam penggunaan sebagai lexer, DFA beroperasi sebagai berikut. Dimulai dari q_0 , setiap karakter a_i dari input menginduksi transisi $\delta(q_{\text{curr}}, a_i) = q_{\text{next}}$. Apabila $q_{\text{curr}} \in F$ dan $\delta(q_{\text{curr}}, a_{i+1})$ tidak terdefinisi atau mengarah ke state error, maka lexeme yang terakumulasi diemisikan sebagai token. Automaton kemudian direset ke q_0 untuk memulai pengenalan token berikutnya. Proses ini berjalan dalam waktu $O(n)$ terhadap panjang input n , karena setiap karakter diproses dalam satu transisi $O(1)$.

2 Design and Implementation

2.1 Overview

Program lexer diimplementasikan dalam C++17 menggunakan pendekatan DFA berbasis pembacaan karakter tunggal. Setiap token dikenali melalui satu karakter per iterasi, di mana state machine memutuskan transisi berdasarkan `currChar` dan, bila diperlukan, satu karakter *lookahead* melalui `f.peek()`. Penggunaan lookahead tunggal tetap konsisten dengan model DFA karena setiap transisi tetap deterministik: tidak ada backtracking dan tidak ada pilihan ganda.

Program terdiri dari tiga komponen yang terpisah: `token.hpp` mendefinisikan enum `ArionToken` dan kelas `Token`; `lexer.hpp/lexer.cpp` mengimplementasikan kelas `Lexer` yang menjalankan DFA; dan `main.cpp` menangani antarmuka I/O serta pemrosesan berkas.

2.2 DFA Design

DFA yang diimplementasikan memiliki state awal q_0 yang berfungsi sebagai titik reset setelah setiap token diemisikan. Dari q_0 , karakter pertama yang terbaca menentukan cabang sub-automaton yang akan dimasuki. Diagram 1 menampilkan gambaran umum transisi dari q_0 ke seluruh state utama. Untuk keterbacaan, token berkarakter tunggal (+, −, *, /, (,), [,], ,, ;, .) masing-masing memiliki transisi langsung dari q_0 ke sebuah *accepting state* dan tidak ditampilkan secara individual pada diagram.

Beberapa pola pengenalan memerlukan perhatian lebih lanjut. Diagram 2 menampilkan detail transisi untuk sub-automaton yang lebih kompleks.

Catatan mengenai keyword. Keyword dan identifier memiliki pola leksikal yang identik: keduanya dimulai dengan huruf dan dapat diikuti kombinasi huruf dan angka. Disambiguasi tidak dilakukan melalui state DFA yang terpisah, melainkan melalui *reserved word lookup* pada `std::unordered_map` setelah seluruh lexeme terbaca oleh q_w . Pendekatan ini ekuivalen secara teoritis karena tabel lookup adalah fungsi total dari string ke kategori token, tanpa memerlukan pencabangan automaton. Selain itu, nilai *lexeme* dipertahankan dalam huruf asli (case-sensitive) untuk identifier, sedangkan keyword tidak menyimpan nilai lexeme.

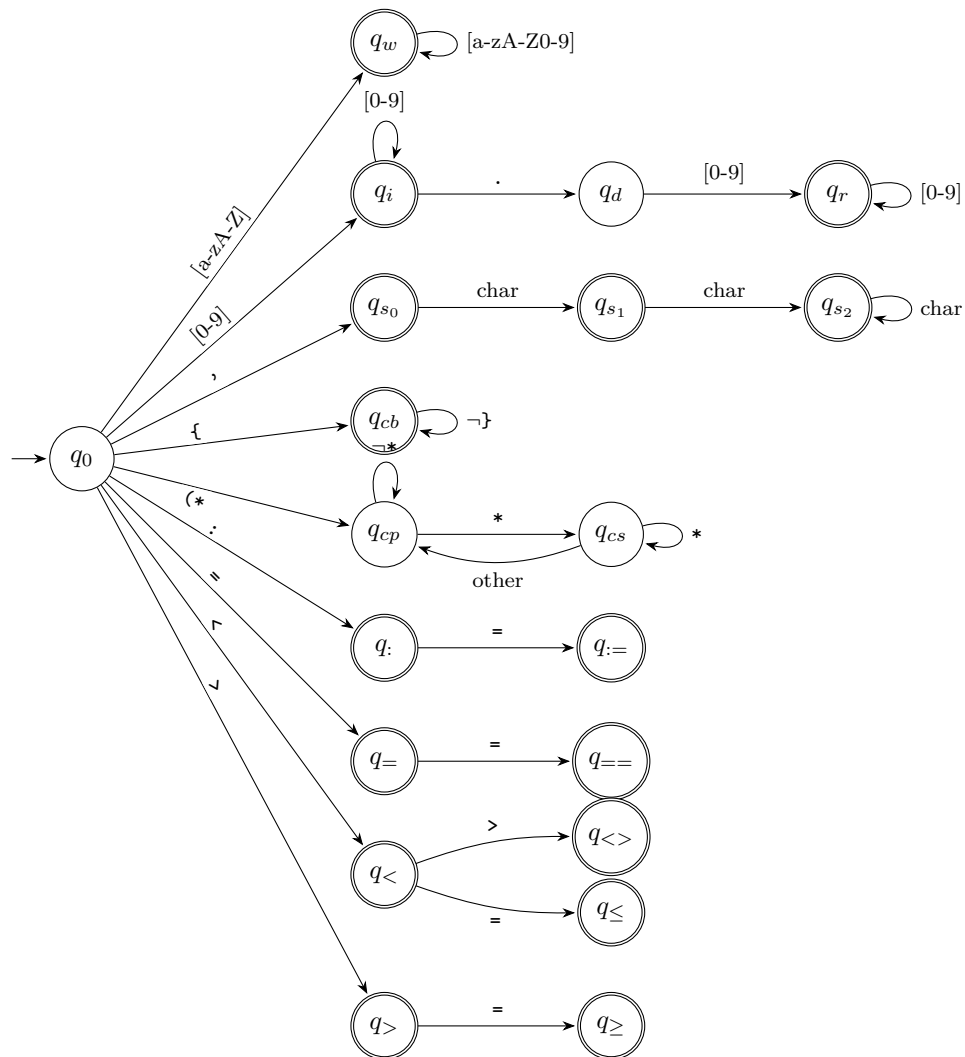
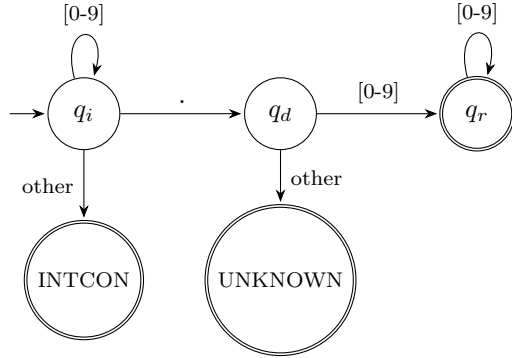
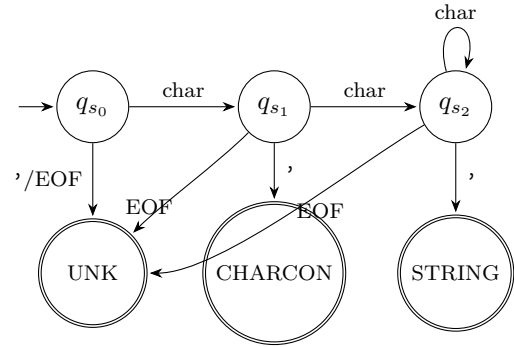


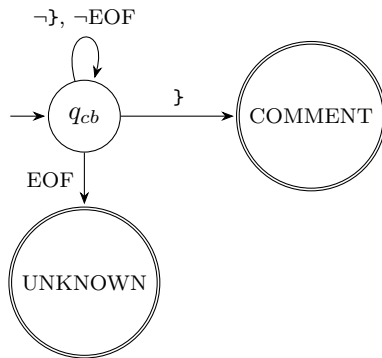
Figure 1: DFA overview: state awal q_0 dan seluruh state utama. State dengan lingkaran ganda adalah *accepting states*. Token berkarakter tunggal tidak ditampilkan individual.



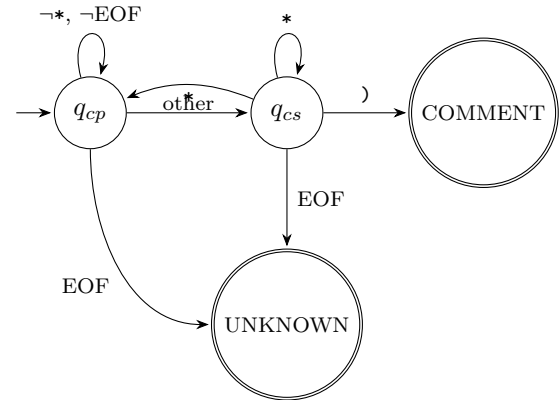
(a) Number recognition



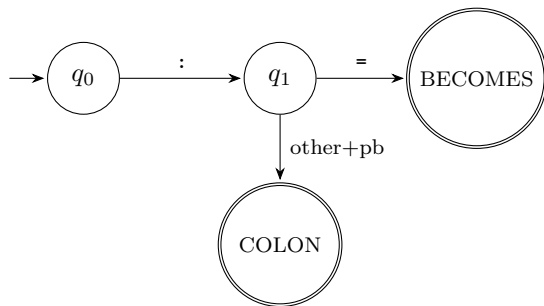
(b) String/char recognition



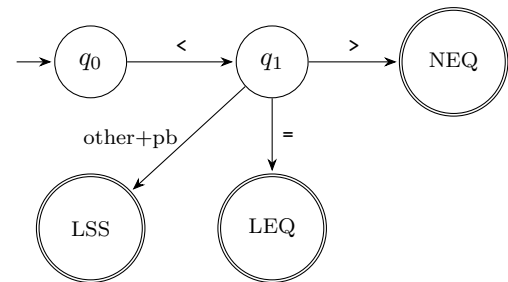
(c) Comment {...} recognition



(d) Comment (*...*) recognition



(e) Colon / Becomes recognition



(f) Relational < operators recognition

Figure 2: Detail sub-automaton untuk pola leksikal kompleks.

2.3 Tokenization Flow

Gambar 3 menunjukkan alur eksekusi fungsi `tokenize()`. Pada setiap iterasi, whitespace dilewati terlebih dahulu, kemudian karakter saat ini digunakan sebagai kondisi dispatch ke sub-automaton yang sesuai. Apabila karakter tidak dikenali oleh pola manapun, token `UNKNOWN` diemisikan sebagai penanda kesalahan leksikal.

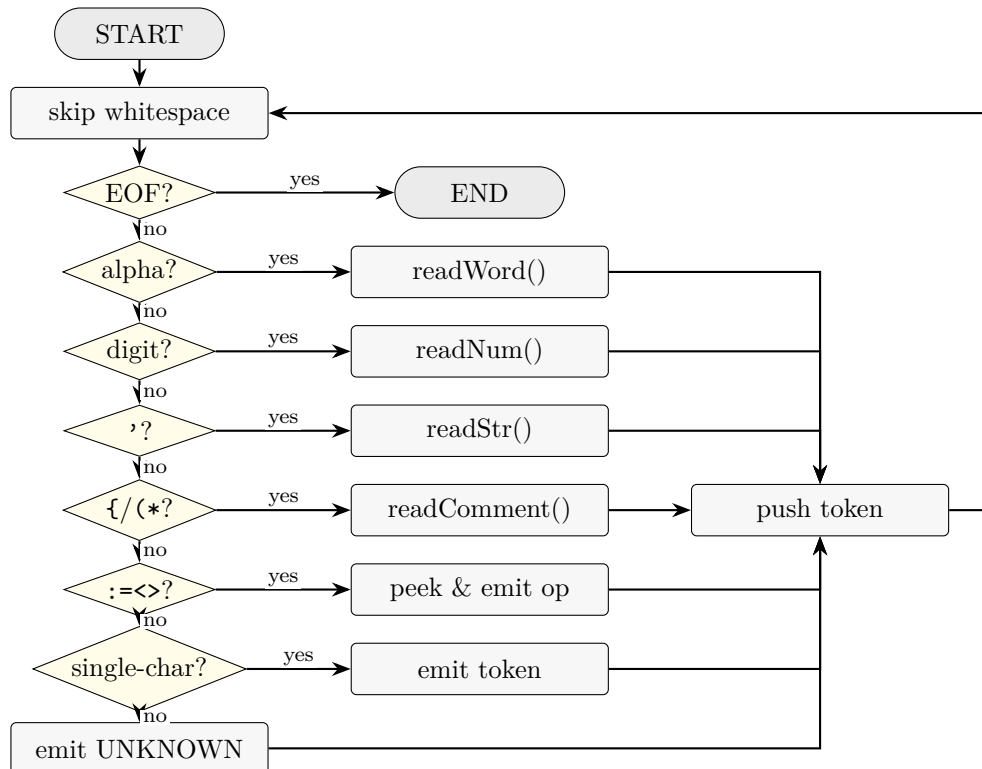


Figure 3: Alur eksekusi fungsi `tokenize()`.

2.4 Token Classification

Tabel 1 merangkum seluruh 53 jenis token yang dikenali oleh lexer beserta pola atau simbol yang merepresentasikannya, dikelompokkan berdasarkan kategori.

Table 1: Daftar token bahasa Arion.

Category	Token	Pattern / Example
<i>Literals</i>		
	<code>intcon</code>	Integer constant: 1, 42, 100
	<code>realcon</code>	Real constant: 3.14, 0.5
	<code>charcon</code>	Single char in quotes: 'A', 'z'
	<code>string</code>	Multi-char in quotes: 'hello', 'IRK'
<i>Arithmetic Operators</i>		
	<code>plus</code>	+
	<code>minus</code>	-
	<code>times</code>	*
	<code>rdiv</code>	/

Continued on next page...

(continued)

Category	Token	Pattern / Example
	idiv	div
	imod	mod
<i>Logical Operators</i>		
	andsy	AND
	orsy	OR
	notsy	NOT
<i>Relational Operators</i>		
	eql	==
	neq	<>
	lss	<
	leq	<=
	gtr	>
	geq	>=
<i>Delimiters & Punctuation</i>		
	lparent	(
	rparent	)
	lbrack	[
	rbrack	]
	comma	,
	semicolon	;
	period	.
	colon	:
	becomes	:=
<i>Declaration Keywords</i>		
	constsy	const
	typesy	type
	varsy	var
	functionsy	function
	proceduresy	procedure
	arraysy	array
	recordsy	record
	programsy	program
<i>Control Flow Keywords</i>		
	beginsy	begin
	endsy	end
	ifsy	if
	thensy	then
	elsesy	else
	casesy	case
	ofsy	of
	whilesy	while
	dosy	do
	repeatsy	repeat
	untilsy	until
	forsy	for
	tosy	to

Continued on next page...

(continued)

Category	Token	Pattern / Example
	downtosy	downto
<i>Special</i>		
	ident	Identifier: <code>x</code> , <code>myVar</code> , <code>h4g</code>
	comment	<code>{...}</code> or <code>(*...*)</code> (discarded)
	unknown	Unrecognized token (lexical error)

3 Testing

4 Conclusion

References

- [1] “Compiler Design - Lexical Analysis,” TutorialsPoint. https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm
- [2] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Pearson, 2006.
- [3] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.
- [4] “An Introduction to Makefiles,” GNU Make Manual. https://www.gnu.org/software/make/manual/html_node/Introduction.html

Appendix

GitHub Repository

<https://github.com/jsndwrld/DCL-Tubes-IF2224-2026>

DFA Transition Diagram

Task Distribution

NIM	Nama	Kontribusi
13524026	Made Branenda Jordhy	
13524028	Muhammad Nur Majiid	
13524034	Jason Edward Salim	
13524068	Athilla Zaidan Zidna Fann	